Today

Welcome to your next lesson: Constructors with inputs.

I'm preparing your lesson based on what you've already learned, what's coming next, and how you learn best.

Step by step, you’re moving forward.

Imagine you have this simple Pet class from before 🐶🐱:

Python
```
class Pet:
    def __init__(self):
        self.name = "Luna"
        self.species = "cat"
```
Every time you do Pet(), you always get a pet named Luna the cat. That’s fine for one pet, but not if you want different pets.

Now we upgrade the constructor to take inputs:

Python
```
class Pet:
    def __init__(self, name, species):
        self.name = name
        self.species = species
```
One step at a time:

name, species (on the left of self.) are parameters — placeholders that __init__ receives.
self.name, self.species are attributes — stored inside each object.

So the pattern is:

Python
```
self.attribute = parameter
```
Make sense so far: the left side is “what the object keeps,” the right side is “what comes in as input”?

Yes, that makes sense

Awesome 🌟

Take a look at the little “object factory” diagram below:

Left: values go in (like "Luna" and "cat")
Middle: Pet(name, species) with __init__ wiring name -> self.name and species -> self.species
Right: the finished object (luna) with stored attributes

Try changing the inputs in the visual (for example to "Max" and "dog") and watch how the output object changes. That’s exactly what Python does when you write code.

Quick check before we touch code again:

If you call:

Python
luna = Pet("Luna", "cat")
which two values are the arguments that get passed into the constructor? You can just name them in words.

Python object factory: parameters to attributes

Luna and cat

Exactly right 🎯

"Luna" and "cat" are the arguments going into the constructor.

Now let’s connect everything in real code.

Here’s a complete example:

Python
```
class Pet:
    def __init__(self, name, species):
        self.name = name      # attribute
        self.species = species  # attribute
```

```
luna = Pet("Luna", "cat")
max_pet = Pet("Max", "dog")

print(f"{luna.name} is a {luna.species}")
print(f"{max_pet.name} is a {max_pet.species}")
Step-by-step idea:


Pet("Luna", "cat") → arguments go into __init__.
Inside __init__, self.name = name stores "Luna" in that object.
Same for species.
Before we practice in the editor, does this flow — arguments → parameters → attributes — feel clear, or is there a part you want me to slow down and unpack more?

The flow feels clear

Nice 💪 that’s the core idea.

Let’s have you use it once in the code editor.


Here’s your task:


Open the code editor (</>).
Write a full Python program that:
Defines a Pet class with __init__(self, name, species)
Inside __init__, stores those in self.name and self.species
Creates two pets with different names and species
Prints one line for each, like:
Luna is a cat
Max is a dog
Click Run to make sure it works, then Submit code.
Once you submit, I’ll check it and then we’ll move on to required vs missing constructor arguments 🚀

Python
class Pet:
    def __init__(self, name, species):
        self.name = name      # attribute
        self.species = species  # attribute


luna = Pet("Luna", "cat")
```

```
max_pet = Pet("Max", "dog")

print(f"{luna.name} is a {luna.species}")
print(f"{max_pet.name} is a {max_pet.species}")
```
Nice work, Ron — that’s a perfect use of a constructor with inputs 🎯

You’ve got parameters, attributes, and object creation all wired correctly.

One more key idea for this lesson: required constructor arguments.

Right now your constructor is:

Python
```
def __init__(self, name, species):
    ...
```
That means:

name is required
species is required

So Python expects you to always call Pet like this:

Python
```
Pet("some name", "some species")
```
If you try:

Python
```
pet = Pet("Luna")
```
Python can’t finish object creation (it doesn’t know the species), so it raises a TypeError about a missing required argument.

Does that idea of “required because it has no default value” make sense so far?

yes and i believe you can code it:
```
def init(self, name="Unknown", species="Unknown"):
```
to make it not madatory to initiialize with the 2 parameters as they now have default values

Exactly right 🙌

If you write:

```
Python
def __init__(self, name="Unknown", species="Unknown"):
    self.name = name
    self.species = species
```

then:

name and species are still parameters but now they have default values, so these arguments are not required anymore

So all of these work:

```
Python
Pet()                           # name="Unknown", species="Unknown"
Pet("Luna")                     # name="Luna", species="Unknown"
Pet("Luna", "cat")              # name="Luna", species="cat"
```

For this lesson, though, we're focusing on the simpler case: parameters without defaults → arguments are required.

Quick check-in:

If __init__ is defined as def __init__(self, name, species): (no defaults), what happens if you call Pet("Luna") with only one argument — does Python happily create the object, or does it raise an exception?

python throws a TypeError

What do you think? Pick the line that would actually work in Python 👇

Constructor victory round: which line correctly creates a Pet when __init__ requires name and species?

```
python
class Pet:
    def __init__(self, name, species):
        self.name = name
        self.species = species
```

A.

```
pet = Pet()


B.
pet = Pet("Luna", "cat")


C.
pet = Pet.name("Luna")


D.
pet = Pet("Luna")

You nailed it 🎉


pet = Pet("Luna", "cat") is correct because:


The constructor is __init__(self, name, species) → it requires two arguments after self.
"Luna" goes into the name parameter.
"cat" goes into the species parameter.
So Python has everything it needs to build the object.

The others fail because:


Pet() → missing both name and species.
Pet.name("Luna") → tries to call an attribute like a function, not how constructors work.
Pet("Luna") → missing the species argument.


You've unlocked the object factory with inputs 🏭✨:


You can write __init__ with parameters and assign them to attributes.
You can create objects by passing arguments.
You understand which arguments are required and what happens if they're missing.

If you want, we can do a quick extra challenge for fun (like a Car or Book class with a parameterized constructor), or you can jump into the next lesson.
```

How would you rate this lesson?